

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

Московский Политехнический Университет

КУРСОВАЯ РАБОТА

Тема: «Разработка интеллектуального чат - бота

для продажи смартфонов»

Вариант 24

по дисциплине «Интеллектуальные системы и технологии»

Выполнил студент:

Хвостенко Дмитрий Борисович

2026 г.

Таблица характеристик чат - бота

В таблице ниже отмечена колонка, на которую претендует данная работа - «100% (отл) + 1 балл на экзамене». Реализованы все требуемые характеристики, включая обработку голоса и дополнительные возможности.

Характеристика	Заявленный уровень
Минимальные функции вопрос - ответ	✓
Расширенный список датасета по намерениям	✓
Наличие датасета диалогов dialogues.txt	✓
Использование API Telegram	✓
Использование сценариев рекламы товара	✓
Количество товаров для рекламы (не менее 3)	✓ (9 шт.)
Обработка голосовых команд и ответ голосом	✓
Другие дополнительные возможности	✓

Претендуемая оценка: 100% (отлично)

Содержание

Введение

1. Теоретические основы обработки естественного языка (NLP)

1.1. Очистка ввода и исправление опечаток (расстояние Левенштейна)

1.2. Лемматизация

1.3. Классификация намерений (intent classification)

1.4. Анализ тональности (sentiment analysis)

1.5. Генерация ответа

2. Машинное обучение для анализа намерений

2.1. Векторизация текста (TF - IDF)

2.2. Классификатор LinearSVC

2.3. Процесс обучения модели

3. Архитектура и реализация чат - бота

3.1. Структура словаря намерений BOT_CONFIG

3.2. Главный алгоритм работы бота

3.3. Контекстное управление темами диалога

3.4. Машина состояний оформления заказа

3.5. Рекламные сценарии и каталог товаров

4. Датасет диалогов и генеративные ответы

5. Голосовой интерфейс (распознавание и синтез речи)

6. Интеграция с мессенджером Telegram

7. Контейнеризация (Docker) и развёртывание (Render)

8. Тестирование

Заключение

Список использованных источников

Введение

В настоящее время диалоговые системы (чат - боты) широко применяются в электронной коммерции для консультирования клиентов, ответов на типовые вопросы и продвижения товаров. Чат-боты позволяют автоматизировать значительную часть общения с покупателями, снижая нагрузку на операторов и повышая скорость обслуживания.

Целью данной курсовой работы является разработка интеллектуального чат-бота, способного вести диалог с пользователем на различные темы и ненавязчиво переводить беседу на рекламу смартфонов с возможностью оформления заказа (вариант 24 - бот для продажи смартфонов).

Для достижения поставленной цели решаются следующие задачи:

- изучение методов обработки естественного языка (NLP);
- реализация классификации намерений пользователя на основе машинного обучения;
- подключение датасета диалогов для генерации ответов на нетиповые вопросы;
- реализация рекламных сценариев и каталога из девяти моделей смартфонов;
- добавление обработки голосовых сообщений (распознавание и синтез речи);
- интеграция бота с мессенджером Telegram;
- контейнеризация приложения и подготовка к облачному развёртыванию.

Программная реализация выполнена на языке Python с использованием библиотек `scikit - learn`, `nltk`, `python - telegram - bot`, `SpeechRecognition`, `gTTS`.

1. Теоретические основы обработки естественного языка (NLP)

Обработка естественного языка (Natural Language Processing, NLP) - то направление искусственного интеллекта, занимающееся анализом и генерацией текста на естественном языке. В разработанном чат - боте обработка пользовательского ввода проходит несколько последовательных этапов.

1.1. Очистка ввода и исправление опечаток (расстояние Левенштейна)

Перед анализом сообщение пользователя очищается: приводится к нижнему регистру, буква «ё» заменяется на «е», удаляются лишние пробелы и символы, не входящие в допустимый алфавит (кириллица, латиница, цифры). Это реализовано в функции `clear_phrase`:

```
def clear_phrase(phrase):  
    phrase = phrase.lower().strip()  
    phrase = re.sub(r'[ёЁ]', 'е', phrase)  
    alphabet = ('абвгде...я' + 'abc...z' + '0123456789 - ')  
    return ''.join(c for c in phrase if c in alphabet).strip()
```

Для устойчивости к опечаткам применяется расстояние Левенштейна - минимальное количество операций вставки, удаления и замены символов, необходимое для преобразования одной строки в другую. Если очищенная фраза пользователя отличается от эталонного примера не более чем на заданную долю символов, она считается совпадающей. Расстояние вычисляется функцией `nltk.edit_distance`.

1.2. Лемматизация

Лемматизация - приведение слова к его начальной (словарной) форме. В работе используется библиотека `Natasha`, решающая базовые задачи обработки

русского языка: сегментацию, морфологический анализ и лемматизацию. Функция `lemmatize_phrase` реализована с «мягкой деградацией»: при отсутствии библиотеки возвращается исходная фраза, что не нарушает работу бота.

1.3. Классификация намерений (intent classification)

Намерение (intent) - это цель высказывания пользователя: приветствие, вопрос о цене, желание купить и т. д. Задача классификации намерений - отнести фразу к одному из заранее определённых классов. В боте определено 33 намерения (приветствие, прощание, выбор по бюджету, карточки товаров, вопросы о камере/батарее/цене, оформление заказа и др.), на которых обучается модель машинного обучения (см. главу 2).

1.4. Анализ тональности (sentiment analysis)

Анализ тональности определяет эмоциональную окраску сообщения. В работе реализован словарный подход: каждому слову из тонального словаря присвоен коэффициент от - 1 (негатив) до +1 (позитив). Тональность фразы вычисляется как среднее значение коэффициентов входящих в неё слов. При сильно негативном тоне (значение ниже -0,3) бот выдаёт сочувственный ответ вместо стандартной заглушки.

1.5. Генерация ответа

Ответ формируется по трёхуровневой схеме: (1) если намерение распознано, выбирается заготовленная реплика; (2) если намерение не распознано, выполняется поиск похожего вопроса в датасете диалогов; (3) если совпадений нет - выдаётся стандартная фраза - заглушка. Дополнительно каждое пятое сообщение сопровождается ненавязчивой рекламной вставкой.

2. Машинное обучение для анализа намерений

2.1. Векторизация текста (TF - IDF)

Алгоритмы машинного обучения работают с числовыми данными, поэтому текст необходимо преобразовать в числовые векторы. Используется метод TF - IDF (Term Frequency - Inverse Document Frequency) на уровне символьных n - грамм длиной 2–3 символа. Символьные n - граммы повышают устойчивость к опечаткам, поскольку слова с ошибками сохраняют большинство общих буквосочетаний.

```
vectorizer = TfidfVectorizer(analyzer='char', ngram_range=(2, 3))  
X = vectorizer.fit_transform(X_text)
```

2.2. Классификатор LinearSVC

Для классификации применяется линейный метод опорных векторов (LinearSVC) из библиотеки scikit - learn. Метод опорных векторов строит гиперплоскости, разделяющие классы в многомерном пространстве признаков, и хорошо подходит для задач классификации текста с большим числом признаков.

```
clf = LinearSVC(max_iter=3000)  
clf.fit(X, y)
```

2.3. Процесс обучения модели

Обучающая выборка формируется из словаря намерений: каждой фразе - примеру сопоставляется метка намерения. В текущей версии модель обучается на 354 примерах, распределённых по 33 классам. После обучения предсказание намерения для новой фразы выполняется методом `clf.predict`, а для повышения точности результат дополнительно проверяется по расстоянию Левенштейна.

3. Архитектура и реализация чат - бота

3.1. Структура словаря намерений BOT_CONFIG

Центральная структура данных - словарь BOT_CONFIG, содержащий намерения. Каждое намерение описывается списком примеров (examples) и списком возможных ответов (responses). Для контекстных намерений задаются дополнительные ключи theme_gen (тема, которую порождает намерение) и

theme_app (темы, в которых намерение применимо).

```
'want_smartphone': {  
    'examples': ['Хочу смартфон', 'Ищу телефон', ...],  
    'responses': ['Какой у вас бюджет?', ...],  
    'theme_gen': 'choosing_smartphone',  
    'theme_app': ['*'],  
}
```

3.2. Главный алгоритм работы бота

Функция bot() реализует основной конвейер обработки: проверка состояния оформления заказа → анализ тональности → классификация намерения → выбор ответа по намерению → генерация из датасета → заглушка → добавление рекламы. Такая последовательность обеспечивает корректную обработку как типовых, так и нестандартных сообщений.

3.3. Контекстное управление темами диалога

Для ведения связного диалога бот хранит историю тем (hist_theme). Намерения могут быть применимы только в определённых темах, что позволяет корректно интерпретировать короткие ответы («да», «нет») в зависимости от контекста предыдущего вопроса. История тем ведётся отдельно для каждого пользователя Telegram.

3.4. Машина состояний оформления заказа

Реализован сценарий оформления заказа. После распознавания намерения «купить» бот запрашивает контактные данные и переводит пользователя в состояние ожидания данных. Следующее сообщение воспринимается как данные заказа, после чего формируется подтверждение с уникальным номером заказа. Предусмотрена возможность отмены оформления.

3.5. Рекламные сценарии и каталог товаров

Каталог содержит девять моделей смартфонов 2026 года в трёх ценовых сегментах:

Модель	Сегмент	Цена, руб.
Samsung Galaxy S26 Ultra	Флагман 2026	от 134 990
iPhone 17 Pro	Флагман 2026	от 129 990
Google Pixel 10 Pro	Флагман 2026	от 99 990
Xiaomi 16 Pro	Флагман 2026	от 89 990
Samsung Galaxy S25 Ultra	Прошлый флагман	от 94 990
iPhone 16 Pro	Прошлый флагман	от 89 990
Samsung Galaxy A56	Средний класс	от 44 990
Redmi Note 14 Pro	Бюджет	от 24 990

Рекламные сценарии реализованы двумя способами: автоматические рекламные вставки (каждое пятое сообщение) и команда /promo с актуальными акциями.

Команды бота в Telegram

Команда	Что делает
/start	Приветствие + список флагманов 2026 в наличии
/help	Справка: что умеет бот
/catalog	Полный каталог смартфонов по сегментам (флагманы / прошлые / средний / бюджет)
/promo	Текущие акции и скидки
/stats	Статистика диалога: сообщений, точность NLU

Помимо команд бот понимает обычные сообщения
Текстом или голосом можно:

- Приветствие/прощание - «привет», «пока»
- Подбор по бюджету - «недорогой», «до 50000», «хочу флагман»
- Карточки товаров - «расскажи об айфоне», «samsung s26», «xiaomi 16 pro», «redmi note 14»
- Характеристики - «какая камера», «долгая батарея», «производительность»
- Цены - «сколько стоит», «прайс»
- Сравнение - «сравни телефоны», «что лучше»
- Покупка - «хочу купить» → ввод данных → подтверждение заказа
- Рассрочка / доставка / акции - «рассрочка», «доставка», «скидки»
- Светская беседа - «расскажи анекдот», «как дела» (ответы из датасета диалогов)

4. Датасет диалогов и генеративные ответы

Для ответов на нетиповые вопросы используется датасет диалогов (dialogues.txt) в формате пар «вопрос - ответ». При загрузке датасет индексируется по словам для ускорения поиска. Когда намерение не распознано, бот ищет наиболее похожий вопрос по расстоянию Левенштейна и возвращает соответствующий ответ. Это придаёт диалогу естественность: на фразы вроде «расскажи анекдот» бот отвечает из датасета, плавно возвращая разговор к теме смартфонов.

5. Голосовой интерфейс (распознавание и синтез речи)

Бот обрабатывает голосовые сообщения. Цепочка обработки: голосовое сообщение (формат OGG) скачивается из Telegram, конвертируется в формат WAV с помощью утилиты ffmpeg, распознаётся в текст через сервис Google (библиотека SpeechRecognition), после чего ответ бота озвучивается с помощью библиотеки gTTS и отправляется пользователю голосовым сообщением.

Каждый этап обработки голоса обёрнут в отдельный блок обработки исключений с подробным логированием, что упрощает диагностику. Для распознавания требуется системная утилита flac, для конвертации - ffmpeg; обе устанавливаются в Docker - образе.

6. Интеграция с мессенджером Telegram

Интеграция выполнена через библиотеку python - telegram - bot (версия 13.x). Зарегистрированы обработчики команд /start, /help, /catalog, /promo, /stats, а также обработчики текстовых и голосовых сообщений. Для каждого пользователя ведётся отдельная история тем и состояние заказа. Сетевые таймауты увеличены для надёжной обработки голосовых сообщений.

7. Контейнеризация (Docker) и развёртывание (Render)

Приложение упаковано в Docker - контейнер на базе образа python:3.11 - slim. В образ устанавливаются системные зависимости (ffmpeg, flac), Python - библиотеки и данные NLTK. Это обеспечивает воспроизводимость и независимость от окружения. Файл docker - compose.yml позволяет запустить бота одной командой.

Для облачного развёртывания на платформе Render подготовлен файл render.yaml, указывающий использовать Docker - рантайм. Поскольку бот работает по принципу опроса (polling) и не принимает HTTP - запросы, в код добавлен вспомогательный HTTP - сервер, удовлетворяющий требованиям веб - сервиса Render.

8. Тестирование

Тестирование проводилось в консольном режиме и в реальном мессенджере Telegram. Проверены следующие сценарии:

- классификация намерений - точность распознавания на тестовых фразах 100 %;
- распознавание моделей, заданных латиницей и с цифрами (iPhone 17 Pro, Redmi Note 14);
- устойчивость к опечаткам (например, «хачу афон» → намерение «iphone»);
- генеративные ответы из датасета диалогов;
- реакция на негативную тональность сообщения;
- полный сценарий оформления заказа с подтверждением;
- обработка голосовых сообщений (распознавание и синтез речи);
- граничные случаи: пустой ввод, бессмысленные длинные фразы.

В ходе тестирования с применением вспомогательных средств анализа кода был выявлен и устранён ряд дефектов: ошибка порядка определения функций, потеря латинских символов при очистке ввода (что мешало распознаванию названий моделей), а также отсутствие системной утилиты `flac` в контейнере, из-за чего не работало распознавание речи. После исправлений все функции работают корректно.

Заключение

В результате выполнения курсовой работы разработан интеллектуальный чат - бот для продажи смартфонов, удовлетворяющий всем требованиям задания, включая обработку голосовых сообщений и дополнительные возможности. Реализованы методы обработки естественного языка: очистка ввода, исправление опечаток на основе расстояния Левенштейна, лемматизация, анализ тональности, классификация намерений с помощью машинного обучения (TF - IDF + LinearSVC) и генерация ответов из датасета диалогов.

Бот ведёт связный диалог с учётом контекста, плавно переводит беседу на рекламу девяти моделей смартфонов, поддерживает оформление заказа и

обрабатывает голосовые сообщения. Приложение контейнеризовано с помощью Docker и подготовлено к развёртыванию в облаке. Поставленные цель и задачи полностью достигнуты.

Список использованных источников

1. Bird S., Klein E., Loper E. Natural Language Processing with Python. - O'Reilly Media, 2009.
2. Документация библиотеки scikit - learn. - URL: [https://scikit - learn.org/](https://scikit-learn.org/)
3. Документация библиотеки NLTK. - URL: <https://www.nltk.org/>
4. Документация python - telegram - bot. - URL: [https://docs.python - telegram - bot.org/](https://docs.python-telegram-bot.org/)
5. Проект Natasha - обработка естественного русского языка. - URL: <https://natasha.github.io/>
6. Документация библиотеки gTTS. - URL: <https://gtts.readthedocs.io/>
7. Документация SpeechRecognition. - URL: <https://pypi.org/project/SpeechRecognition/>
8. Тональный словарь русского языка KartaSlov. - URL: <https://github.com/dkulagin/kartaslov/>
9. Документация Docker. - URL: <https://docs.docker.com/>
10. Документация платформы Render. - URL: <https://render.com/docs/>

Ссылка на бота: <https://t.me/sellphonesssbot>

Ссылка на исходный код: <https://github.com/dimortix/tgbotkursach2026>